

From SGD to AdamW: History, Math, and Mechanisms

Theodore J. LaGrow

College of Computing & College of Lifetime Learning
Georgia Institute of Technology

September 22, 2025

Abstract

This guide builds a principled path from classical stochastic gradient descent (SGD) to AdamW. We start from first principles (empirical risk minimization, minibatch noise, and step-size control), then introduce momentum and Nesterov acceleration as ways to exploit temporal structure in gradients. We move to per-parameter adaptivity (AdaGrad, RMSProp), assemble these ingredients into Adam with bias correction, and finally show why L_2 regularization and “weight decay” are *equivalent* under SGD but *not* under adaptive preconditioning, motivating AdamW’s decoupled update. Throughout, we tie to *Deep Learning* (Goodfellow–Bengio–Courville) §8.3–§8.5, the Adam and AdamW primaries, and the optimization overview of Bottou–Curtis–Nocedal, adding historical notes and practical checklists you can reuse.

1 Problem setup: why optimizers exist

Motivation and historical context. Optimization is the engine under every modern ML model. The move from small, convex problems to deep nonlinear nets forced a shift from exact gradients to noisy minibatch updates, reviving ideas from stochastic approximation (Robbins–Monro, 1951) in the context of backpropagation (Rumelhart–Hinton–Williams, 1986). The practical challenge is not merely *descending* the loss, but doing so in high dimensions with ill-conditioning, noisy gradients, and stringent hardware constraints. Understanding the geometry of noise and curvature explains why we need momentum, adaptive scaling, and decoupled regularization [3, 4, 1, 2].

Context and motivation. Modern ML problems minimize the empirical risk:

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\boldsymbol{\theta}}(x_i), y_i). \quad (1)$$

Here, $f_{\boldsymbol{\theta}}$ is a deep net, ℓ a task loss, and d can be in the millions. Computing $\nabla \mathcal{L}(\boldsymbol{\theta})$ exactly at every step is often infeasible; hence minibatch stochastic gradients:

$$\begin{aligned} \mathbf{g}_t &:= \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\boldsymbol{\theta}} \ell(f_{\boldsymbol{\theta}}(x_i), y_i), \\ \mathbb{E}[\mathbf{g}_t \mid \boldsymbol{\theta}_t] &= \nabla \mathcal{L}(\boldsymbol{\theta}_t). \end{aligned} \quad (2)$$

Modern Machine Learning Perspective (DL §8.3). The choice of optimizer in modern machine learning is shaped by three interlocking facts about modern training:

- (i) **Nonconvex, ill-conditioned landscapes.** Deep networks create loss surfaces with sharp curvature in some directions and flat or even saddle-like regions in others. The Hessian spectrum is typically highly anisotropic, so a single global step size either undershoots along shallow directions or explodes along steep ones. Momentum mitigates zig-zagging by low-pass filtering the update, while preconditioning (diagonal or otherwise) rescales coordinates to better match local curvature. In practice, both ideas target faster descent without violating stability in narrow valleys. See the overview in [1, §8.3] and the large-scale optimization perspective in [2].
- (ii) **Noisy gradients with batch-dependent variance.** SGD uses minibatches to form unbiased stochastic estimates of the full gradient. The variance of this estimate decreases with batch size and depends on the data ordering and the current iterate, which means the algorithm constantly trades exploration (noise can help traverse plateaus and escape saddles) against exploitation (accurate steps near a minimizer). Learning-rate schedules and momentum tune this trade-off over time, while adaptive methods (AdaGrad/RMSProp/Adam) effectively normalize by a running estimate of coordinate-wise second moments to stabilize step magnitudes under heteroskedastic noise. See [1, §8.3–8.5] and [2] for how batch size, noise, and steps interact in theory and practice.
- (iii) **Systems constraints drive large batches and fused updates.** Modern accelerators reward high arithmetic intensity and parallelism: larger batches amortize kernel launches, keep tensor cores busy, and make data-parallel scaling simpler. However, pushing batch size up changes the noise regime (lower variance, different dynamics) and puts pressure on optimizers to remain stable and sample-efficient at high throughput. This is why practical recipes pair large-batch training with warmup, decay schedules, and optimizers that behave predictably under different noise levels. Fusing update rules (e.g., maintaining EMAs for moments) also reduces overhead and cache misses. See [1, §8.3] for a more practical viewpoint and [2] for systems-aware guidance.

Bottom line. A practical optimizer must balance *progress* (taking confident steps along low-curvature directions) with *stability* (not overshooting along steep or noisy directions). Momentum provides temporal averaging; adaptive preconditioning provides coordinate-wise normalization; and decoupled weight decay (AdamW) restores a clean capacity control that is otherwise tangled with adaptivity. Schedules (warmup and decay) modulate this balance over the course of training [1, 2, §8.3–8.5].

Noise geometry. Write the stochastic gradient as $\mathbf{g}_t = \nabla \mathcal{L}(\boldsymbol{\theta}_t) + \boldsymbol{\xi}_t$, with $\mathbb{E}[\boldsymbol{\xi}_t] = 0$ and covariance Σ_t . With a constant step size α , iterates execute a noisy gradient flow and, in convex/quadratic settings, converge to a steady-state distribution whose spread scales with α and Σ_t [2]. This motivates *averaging* (momentum) and *preconditioning* (adaptive methods).

2 SGD to Momentum to Nesterov

Motivation and historical context. Plain SGD can be painfully slow in ravines: directions of wildly different curvature create zig-zagging that wastes compute. Historically, momentum (Polyak, 1964) imported a physics-inspired idea (velocity) into optimization, while Nesterov (1983) introduced a look-ahead correction that achieves provably faster rates in convex problems. In modern machine

learning, these tools improve stability and wall-clock to a target loss by exploiting temporal structure in gradients [5, 6, 1].

2.1 Plain (stochastic) gradient descent

The baseline update is

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha_t \mathbf{g}_t. \quad (3)$$

With $\alpha_t = \alpha$ constant, SGD can *jitter* across level sets when gradients vary in magnitude across coordinates (ravines). Decaying schedules $\alpha_t \downarrow$ can guarantee convergence in convex problems, but overly aggressive decay stalls progress in deep nets [1, §8.3].

2.2 Heavy-ball momentum (Polyak, 1964)

Momentum introduces a velocity $\mathbf{v}_t$ that exponentially averages recent gradients:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \mathbf{g}_t, \quad 0 \leq \beta < 1, \quad (4)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \mathbf{v}_{t+1}. \quad (5)$$

Why it helps. In directions where the gradient retains the same sign, $\mathbf{v}_t$ amplifies signal; in directions where it alternates, the average damps noise. In ill-conditioned basins, momentum effectively *stretches* steps along low-curvature directions and *shrinks* steps along high-curvature ones.

2.3 Nesterov’s accelerated gradient (NAG)

NAG evaluates the gradient *at a look-ahead point*:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \nabla \mathcal{L}(\boldsymbol{\theta}_t - \alpha \beta \mathbf{v}_t), \quad (6)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \mathbf{v}_{t+1}. \quad (7)$$

In convex settings, with tailored schedules, NAG achieves a rate $\mathcal{O}(1/t^2)$, improving over $\mathcal{O}(1/t)$, and in deep nets it often produces smoother trajectories in curved regions [1, 6, §8.3].

Intuition Map: Noisy Descent and Optimizer Design

Setup. Training a deep net is like descending an unknown mountain in fog. You have a noisy local direction, limited budget, and choices about how to smooth noise and pace steps. The table links the intuition to the math you use elsewhere:

Intuition	Formal counterpart
Position	parameters θ
Altitude	objective $\mathcal{L}(\theta)$
Compass reading	stochastic gradient g
Inertia	momentum (e.g., β)
Controller / rescaling	preconditioner D_t^{-1} (diagonal)
Pack shedding	capacity control: weight decay vs. L_2
Safety leash	gradient clipping
Paved path	variance reduction / better conditioning
Pace plan	learning-rate schedule (warmup/decay)
Convoy size	batch size $ B $ (noise level)

Optimizers as “loadouts.”

- **SGD.** Step where the compass points now. Simple and effective, but jittery in ravines (ill-conditioning); progress depends heavily on schedule.
- **Momentum (heavy-ball).** Temporal averaging adds inertia: amplifies consistent directions, damps oscillations; can overshoot on sharp turns.
- **Nesterov (look-ahead).** Peek at the point momentum will take you, then correct; reduces overshoot without losing glide.
- **AdaGrad.** Per-coordinate steps shrink with cumulative squared gradients; great for sparse/rare features; steps can become too small late.
- **RMSPprop.** Uses a *recent* (EMA) second-moment estimate; forgets old curvature so it adapts to nonstationary landscapes.
- **Adam.** Momentum + RMSPprop with bias correction so early steps have the right scale; fast to reduce loss, generalization varies by task.

L_2 vs. weight decay (why AdamW).

- Under *SGD*, adding L_2 penalty is equivalent to multiplicative shrinkage (weight decay) each step.
- Under *adaptive* preconditioning, L_2 is scaled by D_t^{-1} , so shrinkage depends on gradient history (coupled, uneven).
- **AdamW** decouples decay: apply uniform shrinkage to θ *separately* from the adaptive step, restoring a clean capacity knob.

Design implications (quick rules).

- Use *momentum* for temporal averaging; use *adaptive scaling* when gradient magnitudes are highly anisotropic/heteroskedastic.
- Keep *capacity control* independent of stabilization: prefer decoupled decay (AdamW) over L_2 inside adaptives.
- Treat schedules as first-class: warmup for large batches; cosine/step decay to land softly; clip gradients to prevent rare blow-ups.
- Remember batch-size tradeoffs: larger $|B|$ lowers noise (steadier steps) but increases inertia and can hurt exploration—adjust α and warmup.
- Invest in “paved paths”: normalization, data pipelines, and averaging reduce variance so simpler optimizers (e.g., SGD+momentum) can excel.

3 Per-parameter adaptivity: AdaGrad and RMSProp

Motivation and historical context. A single global learning rate is a blunt instrument in high dimensions. The online-learning era (circa 2010–2012) revealed that rescaling each coordinate by its own historical gradient statistics accelerates training, especially with sparse features. AdaGrad (2011) made this precise; RMSProp (2012) replaced cumulative memory with an exponential one, fitting nonconvex deep nets better [7, 8, 9, 1].

3.1 AdaGrad (online learning origin)

AdaGrad keeps a running sum of squared gradients,

$$G_t = \sum_{\tau=1}^t \mathbf{g}_\tau \odot \mathbf{g}_\tau,$$

and uses the diagonal preconditioner $(\sqrt{G_t} + \epsilon)^{-1}$, shrinking steps for coordinates with sustained large gradients and allowing larger steps for rare/infrequently-updated features. The downside is *irreversible shrinkage*: G_t only grows, so learning rates can become vanishingly small [1, §8.5].

3.2 RMSProp (exponential memory)

RMSProp replaces cumulative sums with an exponential moving average (EMA):

$$\mathbf{v}_{t+1} = \rho \mathbf{v}_t + (1 - \rho) \mathbf{g}_t \odot \mathbf{g}_t, \quad (8)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \frac{\mathbf{g}_t}{\sqrt{\mathbf{v}_{t+1}} + \epsilon}. \quad (9)$$

This avoids AdaGrad’s monotone decay by letting old curvature information *fade*, striking a practical balance in nonconvex landscapes [8, 9]. Hyperparameters $\rho \approx 0.9$ – 0.99 and ϵ in 10^{-8} – 10^{-7} are common.

4 Adam: moments with bias correction

Motivation and historical context. Adam (2015) unified momentum and RMSProp in a way that *just works* across many workloads, explaining its decade-long popularity. Its key innovation is not only using first and second moment EMAs but also correcting their initial bias, a small detail with large early-time effects. Adam sits among contemporaries like AdaDelta and Nadam, but remains the baseline many compare against [11, 10, 1].

Design and update rule

Adam blends (i) momentum on gradients (first moment) and (ii) RMSProp-style scaling (second moment). Critically, it *bias-corrects* the EMAs because they start at zero:

$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}_t, \quad \hat{\mathbf{m}}_{t+1} = \frac{\mathbf{m}_{t+1}}{1 - \beta_1^{t+1}}, \quad (10)$$

$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t, \quad \hat{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1 - \beta_2^{t+1}}, \quad (11)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \frac{\hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1}} + \epsilon}. \quad (12)$$

Why bias correction matters (early steps). Without correction, $\mathbb{E}[\mathbf{m}_t]$ and $\mathbb{E}[\mathbf{v}_t]$ are scaled down by $(1 - \beta_1^t)$ and $(1 - \beta_2^t)$ due to zero initialization; with large (β_1, β_2) , early steps shrink artificially. Correcting by $(1 - \beta_1^t)^{-1}$ and $(1 - \beta_2^t)^{-1}$ restores the intended scale [11].

Practice notes (DL §8.5). Defaults $(\beta_1, \beta_2, \epsilon) = (0.9, 0.999, 10^{-8})$ work well; warmup for α , cosine/step decay, and gradient clipping help in very deep nets. Adam often wins on speed-to-target but does not always generalize best; see [12].

5 L_2 vs. weight decay: equivalence and inequivalence

Motivation and historical context. Regularization predates deep learning: Tikhonov shrinkage goes back to the 1940s, and “weight decay” has been a staple of neural nets since at least the early 1990s [13, 14, 1]. For SGD, L_2 penalty and multiplicative decay are two faces of the same coin; under adaptive preconditioning they diverge in subtle but important ways. Understanding this is key to making fair comparisons and motivated the AdamW update [15].

Classical equivalence under SGD

Let $R(\boldsymbol{\theta}) = \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2$. Minimizing $\mathcal{L}(\boldsymbol{\theta}) + R(\boldsymbol{\theta})$ with SGD gives

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha(\nabla \mathcal{L}(\boldsymbol{\theta}_t) + \lambda \boldsymbol{\theta}_t) = (1 - \alpha\lambda) \boldsymbol{\theta}_t - \alpha \nabla \mathcal{L}(\boldsymbol{\theta}_t). \quad (13)$$

Thus L_2 and weight decay are interchangeable for SGD (or any method with scalar preconditioner).

Breakdown under adaptive preconditioning. Adaptive methods use a diagonal preconditioner D_t^{-1} :

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha D_t^{-1}(\nabla \mathcal{L}(\boldsymbol{\theta}_t) + \lambda \boldsymbol{\theta}_t). \quad (14)$$

Now shrinkage is coordinate-wise $\alpha\lambda(D_t^{-1}\boldsymbol{\theta}_t)_j$, coupling regularization to gradient history. L_2 no longer acts like uniform decay.

Proposition 1 (Equivalence criterion). Suppose an optimizer updates $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha P_t(\nabla \mathcal{L}(\boldsymbol{\theta}_t) + \lambda \boldsymbol{\theta}_t)$ with SPD P_t . The update is equivalent to a weight-decay-plus-plain-step rule (i.e., $\boldsymbol{\theta}_{t+1} = (1 - \alpha\lambda)\boldsymbol{\theta}_t - \alpha P_t \nabla \mathcal{L}(\boldsymbol{\theta}_t)$) iff $P_t = c_t \mathbf{I}$ for some $c_t > 0$.

Sketch. Equivalence requires $P_t \boldsymbol{\theta} = \boldsymbol{\theta}$ for all $\boldsymbol{\theta}$ (up to scalar rescaling of α), forcing P_t to be a multiple of $\mathbf{I}$. $\square$

6 AdamW: decoupled weight decay

Motivation and historical context. By 2017, practitioners noticed that “Adam + L_2 ” was not behaving like the intended weight shrinkage, especially in highly anisotropic regimes common in vision and NLP. AdamW formalized the fix: decouple decay from the adaptive step so that λ regains its classical, architecture-agnostic meaning. This clarified many empirical results and became a widely adopted default in large-scale training [15].

The decoupled update

AdamW applies classical shrinkage *outside* of the adaptive step:

$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t, \quad (15)$$

$$\hat{\mathbf{m}}_{t+1} = \frac{\mathbf{m}_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1 - \beta_2^{t+1}}, \quad (16)$$

$$\boldsymbol{\theta}_{t+1} = (1 - \alpha\lambda) \boldsymbol{\theta}_t - \alpha \frac{\hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1} + \epsilon}}. \quad (17)$$

Decoupling restores a clean capacity-control knob and simplifies tuning: set α for step size and λ for shrinkage without interference [15].

When it helps (empirical pattern). On problems with anisotropic gradients, Adam+L₂ can under-regularize some coordinates and over-regularize others. AdamW removes this distortion, often narrowing seed-to-seed variability and improving test performance at matched compute. Adam may still win on speed-to-training-loss, while generalization sometimes favors SGD+momentum; always validate at matched budgets [12].

Remark 1 (Library footnote). Many frameworks provide both Adam and AdamW. Ensure that the “weight_decay” argument in Adam uses L₂-style coupling (baseline) rather than decoupled semantics; otherwise your “Adam” baseline behaves like AdamW.

7 Hyperparameters in Practice

Orientation. Adam/AdamW expose few knobs, but they interact. Think of α (step size) setting the *pace*, (β_1, β_2) setting the *memory*, ϵ setting a *floor* for scaling, and λ (decoupled decay) setting *capacity control*. Batch size, warmup, decay schedules, clipping, and parameter grouping complete the picture [1, 2, 11, 15, §8.3–8.5].

Core knobs (Adam/AdamW)

- **Learning rate α .** Controls overall step size after normalization by $\sqrt{\hat{\mathbf{v}}} + \epsilon$. *Too high*: loss spikes, exploding updates, oscillations; *too low*: slow, under-fitting. With Adam/AdamW, α is less sensitive to gradient scale than SGD but remains the dominant stability lever.
- **Momentum β_1 (first moment EMA).** Default $\beta_1 \approx 0.9$ accumulates direction; higher values smooth more but react slower to trend changes. *Increase* for very noisy tasks; *decrease* to improve agility in rapidly changing regimes (e.g., curriculum shifts).
- **Second-moment β_2 (curvature/noise EMA).** Default $\beta_2 \approx 0.999$ stabilizes denominators. *Lowering* to 0.995–0.99 reacts faster (useful with tiny batches / nonstationary data) but can destabilize; *raising* to 0.9995 reduces denominator noise but prolongs early under-scaling without adequate bias correction.
- **Epsilon ϵ .** Added inside the denominator to avoid division by a tiny $\sqrt{\hat{\mathbf{v}}}$. Typical 10^{-8} – 10^{-6} . Larger ϵ dampens adaptivity (steps behave more like momentum SGD); smaller ϵ increases sensitivity to $\hat{\mathbf{v}}$ noise and can cause jitter when gradients are near-zero. Keep it fixed unless you diagnose denominator pathologies.

- **Weight decay λ (decoupled).** In AdamW, effective per-step shrink is approximately $(1 - \alpha\lambda)$, so the *per-epoch* effect scales with step count. If you change batch size or the number of steps per epoch, adjust λ to keep epoch-level shrink roughly constant (rule-of-thumb: maintain $\alpha\lambda$ when you only scale α ; scale λ inversely with steps per epoch if you change batch size or accumulation).

Training-loop knobs (schedule, batch, clipping, grouping)

- **Warmup.** 200–1000 steps (or $\sim 1\text{--}3\%$ of total steps) prevents early overshoot when EMAs are still calibrating. Larger warmup is especially helpful with large batches or high β_2 .
- **Decay schedule.** Cosine or step decay reduces α as you approach minima, trading speed for stability and generalization. If you halve the number of decay steps, consider doubling λ to keep total shrink comparable.
- **Batch size $|B|$.** Larger $|B|$ lowers gradient variance (steadier steps) but reduces exploration and increases the risk of sharp-minimum convergence; often requires LR warmup and slightly *lower* peak α in Adam/AdamW compared to naive linear scaling.
- **Gradient clipping.** Global-norm clipping at 0.5–1.0 is a safety valve for rare spikes; clips *before* the Adam/AdamW transform to avoid distorting preconditioning statistics.
- **Parameter groups.** Apply decay to weights only; *exclude* biases and normalization scale/shift parameters from decay (`weight_decay=0` for those). Optionally use smaller α for very sensitive layers (e.g., embeddings) or layer-wise LR decay in deep stacks.

How each knob “feels” when you turn it

Knob	Typical	Symptoms $\rightarrow$ Adjustment
α	$10^{-4}\text{--}3 \times 10^{-3}$	<i>Divergence / spiky loss</i> $\Rightarrow \downarrow \alpha$ or add warmup; <i>stalled loss</i> $\Rightarrow \uparrow \alpha$ or relax decay; <i>noisy plateau</i> $\Rightarrow$ keep α , $\uparrow \beta_1$ or $\uparrow \beta_2$.
β_1	0.8–0.95	<i>Zig-zag, high update variance</i> $\Rightarrow \uparrow \beta_1$; <i>slow to follow regime change</i> $\Rightarrow \downarrow \beta_1$.
β_2	0.99–0.9995	<i>Denominator too sluggish early</i> $\Rightarrow$ shorter warmup or $\downarrow \beta_2$; <i>denominator noise / chattery steps</i> $\Rightarrow \uparrow \beta_2$ or $\uparrow \epsilon$.
ϵ	$10^{-8}\text{--}10^{-6}$	<i>Tiny gradients cause jumps</i> $\Rightarrow \uparrow \epsilon$; <i>adaptivity feels “muted”</i> $\Rightarrow \downarrow \epsilon$ (carefully).
λ (AdamW)	$10^{-6}\text{--}10^{-2}$	<i>Overfitting / sharp minima</i> $\Rightarrow \uparrow \lambda$; <i>underfitting</i> $\Rightarrow \downarrow \lambda$; <i>changed steps/epoch</i> $\Rightarrow$ rescale λ to keep epoch-level shrink.
Warmup	200–1000 steps	<i>Early loss spike / EMA bias</i> $\Rightarrow \uparrow$ warmup; <i>need speed</i> $\Rightarrow \downarrow$ warmup (if stable).
Clip norm	0.5–1.0	<i>Rare catastrophic jumps</i> $\Rightarrow$ enable/raise; <i>chronic over-clipping</i> $\Rightarrow$ lower α or increase batch.

Interactions and invariances (what changes together)

- **α and λ couple via steps.** With decoupled decay, the per-step shrink $\approx (1 - \alpha\lambda)$ means *total* shrink over T steps is $(1 - \alpha\lambda)^T$. If you double steps per epoch (smaller batches), halve λ to keep epoch-level shrink similar.

- **β_2 vs. warmup.** Higher β_2 (e.g., 0.9995) needs longer warmup to avoid under-scaled early updates; lower β_2 tolerates shorter warmup but can jitter late.
- **ϵ position matters.** The classic Adam/AdamW place ϵ *inside* the denominator. Moving it outside changes the effective floor; stick to the standard unless reproducing a specific paper/repo.
- **Adam vs. AMSGrad.** AMSGrad enforces a non-decreasing second-moment estimate. It can help in rare nonstationary failures but is usually unnecessary with good schedules and clipping.

A pragmatic tuning path

1. Fix dataset, augmentation, batch size (as large as fits), and apply clipping at 1.0.
2. Run a quick LR-range test to bracket α ; pick the largest value that yields smooth, monotone loss for a few hundred steps.
3. Set $(\beta_1, \beta_2, \epsilon) = (0.9, 0.999, 10^{-8})$; if batches are very small/noisy, try $\beta_2 = 0.99$.
4. Choose a decay schedule (cosine/step) and warmup (1–3% of total steps).
5. Tune λ with validation: start at 10^{-4} (dense vision) or 0 – 10^{-5} (very small models), then adjust; rescale λ if you later change steps per epoch.

Rule of thumb. Track the update-to-parameter ratio $\|\Delta\theta\|/\|\theta\|$ per step; values in 10^{-3} – 10^{-2} during mid-training often correlate with healthy progress. If it collapses early, raise α or shorten warmup; if it spikes unpredictably, lower α or increase clipping.

8 Pseudocode (reference)

Motivation and historical context. Having the updates in one place reduces implementation bugs during ablation studies and clarifies where decoupling happens. These references mirror the notation above for quick comparison across methods.

Algorithm 1 SGD / Momentum / Nesterov (mini-batch)

- 1: **Input:** $\alpha > 0$, $\beta \in [0, 1)$, batch size b , epochs T
 - 2: initialize θ_0 , $\mathbf{v}_0 = \mathbf{0}$
 - 3: **for** $t = 0, \dots, T - 1$ **do**
 - 4: sample B_t of size b , compute $\mathbf{g}_t = \frac{1}{b} \sum_{i \in B_t} \nabla \theta \ell(f_{\theta}(x_i), y_i)$
 - 5: **(SGD)** $\theta_{t+1} = \theta_t - \alpha \mathbf{g}_t$
 - 6: **(Momentum)** $\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \mathbf{g}_t$, $\theta_{t+1} = \theta_t - \alpha \mathbf{v}_{t+1}$
 - 7: **(NAG)** $\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \nabla \mathcal{L}(\theta_t - \alpha \beta \mathbf{v}_t)$, $\theta_{t+1} = \theta_t - \alpha \mathbf{v}_{t+1}$
 - 8: **end for**
-

Algorithm 2 Adam and AdamW (mini-batch)

```
1: Input:  $\alpha > 0, \beta_1, \beta_2 \in [0, 1), \epsilon > 0$ , decay  $\lambda \geq 0$ 
2: initialize  $\theta_0, \mathbf{m}_0 = \mathbf{0}, \mathbf{v}_0 = \mathbf{0}$ 
3: for  $t = 0, \dots, T - 1$  do
4:   sample  $B_t$ , compute  $\mathbf{g}_t$ 
5:    $\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t$ 
6:    $\hat{\mathbf{m}}_{t+1} = \mathbf{m}_{t+1} / (1 - \beta_1^{t+1}), \quad \hat{\mathbf{v}}_{t+1} = \mathbf{v}_{t+1} / (1 - \beta_2^{t+1})$ 
7:   Adam step:  $\theta_{t+1} = \theta_t - \alpha \hat{\mathbf{m}}_{t+1} / (\sqrt{\hat{\mathbf{v}}_{t+1}} + \epsilon)$ 
8:   AdamW decay:  $\theta_{t+1} \leftarrow (1 - \alpha\lambda) \theta_{t+1}$  (decoupled)
9: end for
```

9 Practical checklist and failure modes

Motivation and historical context. Fair comparisons live or die on the details: mismatched batches or schedules can swamp optimizer effects. A short checklist prevents confounds and makes results reproducible across seeds and hardware [1, 2, 12].

Before you run. Fix splits, batch size, data augmentation, seeds, and hardware class across optimizers. Pre-commit a loss target ℓ that most methods can reach and a compute budget (gradient evaluations and wall-clock).

During.

- For Adam vs. AdamW, *do not* emulate decay in Adam with L_2 and call it weight decay; use explicit decoupled decay for AdamW.
- Track and report stability (median $\pm$ IQR across seeds), loss vs. time/evals, and test metrics at matched compute.
- Try small LR grids rather than trusting defaults; couple α with batch size (linear scaling is a decent first guess).
- Consider warmup for Adam/AdamW; use cosine or step decay thereafter.

Common failure modes. Divergence at large α (mark cells as divergent, do not delete), “lucky seed” over-interpretation (always aggregate), and confounding schedule/batch changes across optimizers (invalidates time-to- ℓ comparisons).

Acknowledgments and further reading

Deep Learning (DL §8.3–§8.5) frames SGD, momentum, and adaptive methods with practical guidance [1]. **Bottou–Curtis–Nocedal** overview optimization at scale, explaining noise, curvature, and schedules [2]. **Adam** introduced bias-corrected moment adaptivity [11]. **AdamW** clarified why L_2 and decay diverge under adaptivity and proposed the decoupled update [15]. For a perspective on generalization with adaptives vs. SGD, see **Wilson et al.** [12].

References

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Chapter 8: Optimization for Training Deep Models. Online: <https://www.deeplearningbook.org/contents/optimization.html>.
- [2] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization Methods for Large-Scale Machine Learning. *SIAM Review*, 60(2):223–311, 2018.
- [3] Herbert Robbins and Sutton Monro. A Stochastic Approximation Method. *Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [4] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- [5] Boris Polyak. Some Methods of Speeding up the Convergence of Iteration Methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
- [6] Yurii Nesterov. A Method for Solving the Convex Programming Problem with Convergence Rate $\mathcal{O}(1/k^2)$. *Soviet Math. Dokl.*, 27:372–376, 1983.
- [7] John Duchi, Elad Hazan, and Yoram Singer. Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.
- [8] Geoffrey Hinton. Neural Networks for Machine Learning, Lecture 6a/6.5: rmsprop (Coursera), 2012. Slides/video notes.
- [9] Tijmen Tieleman and Geoffrey Hinton. RMSProp and equilibrated adaptive learning rates for non-convex optimization. Technical note, 2015.
- [10] Matthew D. Zeiler. ADADELTA: An Adaptive Learning Rate Method. arXiv:1212.5701, 2012.
- [11] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015. arXiv:1412.6980.
- [12] Ashia C. Wilson, Rebecca Roelofs, Mitchell Stern, Nati Srebro, and Benjamin Recht. The Marginal Value of Adaptive Gradient Methods in Machine Learning. *NeurIPS*, 2017.
- [13] Andrey N. Tikhonov. On the stability of inverse problems. *Doklady Akademii Nauk SSSR*, 39(5):195–198, 1943.
- [14] Anders Krogh and John A. Hertz. A Simple Weight Decay Can Improve Generalization. *Advances in Neural Information Processing Systems (NIPS)*, 1992.
- [15] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. *ICLR*, 2019. arXiv:1711.05101.

10 Version Control

- v1.0 - 09/22/2025 - TJL published the guide, 'From SGD to AdamW' to the CS7641 Fall 2025 course.